Breaking the Ratchet

CS6008 —

Assignment 3

Roll numbers: 24M0818, 24M0816

Overview

This report covers CS6008 Assignment 3 (Breaking the Ratchet), a Signal-style X3DH + Double Ratchet secure-messaging lab (X25519 DH, Ed25519 signatures, HKDF-SHA256/HMAC-SHA256 KDFs, ChaCha20-Poly1305 AEAD) delivered as newline-delimited JSON over raw TCP, run as a multi-container Docker Compose lab (s4ch1n/as3:latest, OPK_COUNT=5). Three attacks were built, each matching one implementation-level weakness the lab exposes, and verified end-to-end against the real, unmodified server and client binaries — not a simulation. Phase 1 establishes a healthy, unmodified baseline conversation between Alice and Bob so no later attack result can be blamed on a chain that was already broken. Attack 0 shows that the receiver does real cryptographic and memory work on a forged message's header before the AEAD tag is ever checked, letting an on-path attacker burn CPU and memory with messages that are never actually accepted. Attack 1 shows that one-time prekeys are consumed merely by being requested, not by actually being used, letting any registered user silently exhaust a victim's prekey inventory and block new sessions with them. Attack 2 shows that because the relay server is also the only source of identity keys, with no out-of-band or persisted check, a malicious server can transparently substitute forged identities for both parties and read every message in both directions while both victims see a completely ordinary conversation.

Every section below follows the same shape: what the attack targets and why it works (derived from protocol.md and the client/server source), the attack tool itself, real evidence pulled from this repository's evidence/ directories, and a proposed fix. A closing appendix documents how Claude was used throughout, including a detailed account of debugging Attack 2's one real implementation bug.

Phase 1 — Build and Baseline Verification

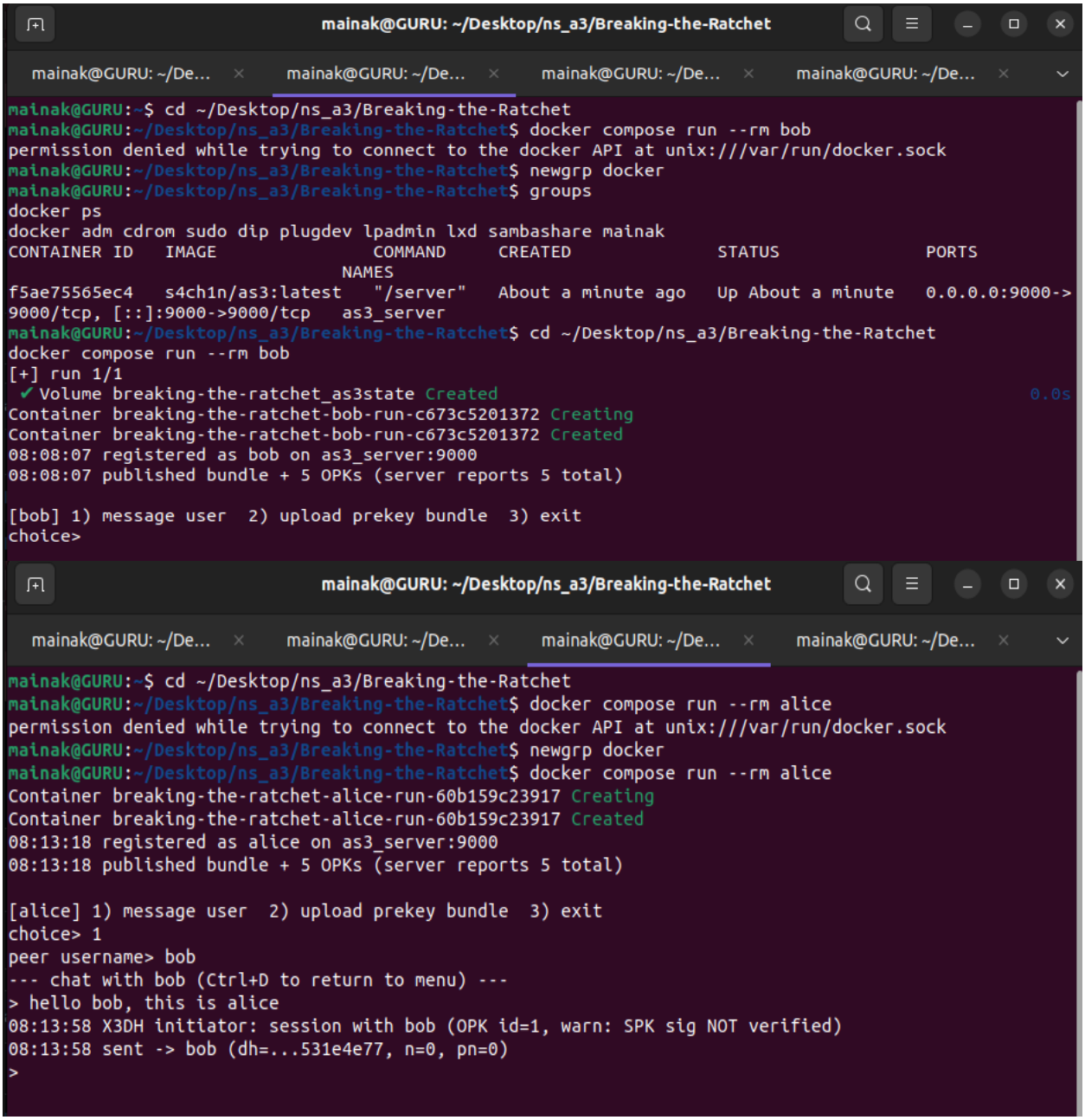
Before any attack was mounted, the unmodified lab (real as3_server, real Alice and Bob clients) was brought up from a clean Docker state and a full two-way conversation was exchanged directly through the genuine server, to establish that the environment itself is healthy and that every later attack result can be attributed to the attack, not to an already-broken baseline.

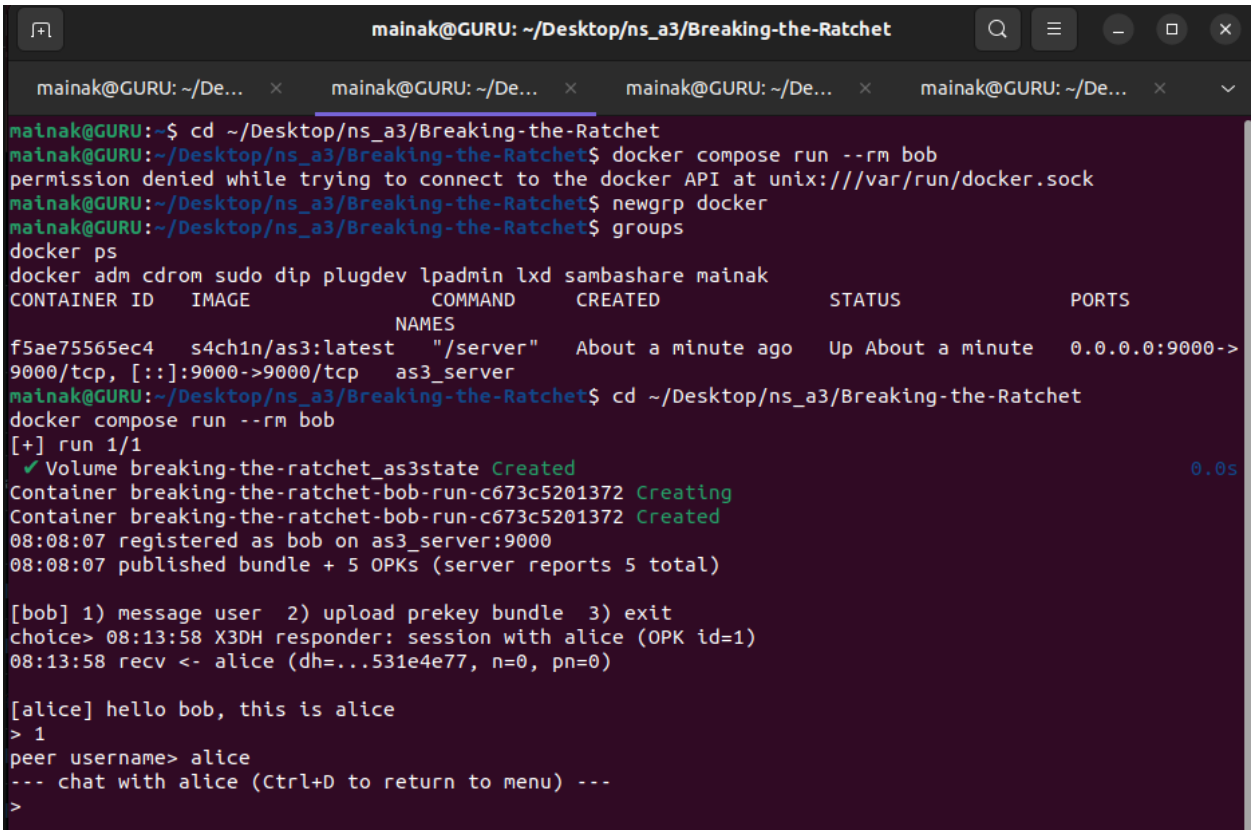
What was verified

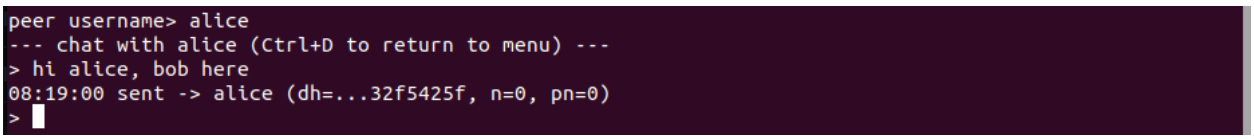
- Bob registers, publishes his identity/signed-prekey/one-time-prekey bundle, and the server acknowledges with the correct OPK count.
- Alice performs a genuine X3DH handshake against Bob's real bundle and sends message A1; Bob's client decrypts it correctly with no warnings.
- Bob replies with message B1 on the same ratchet session; Alice's client decrypts it correctly.
- The relay server's own log shows it only ever handles opaque ciphertext — it never has, and never prints, the plaintext of a message it is honestly relaying.

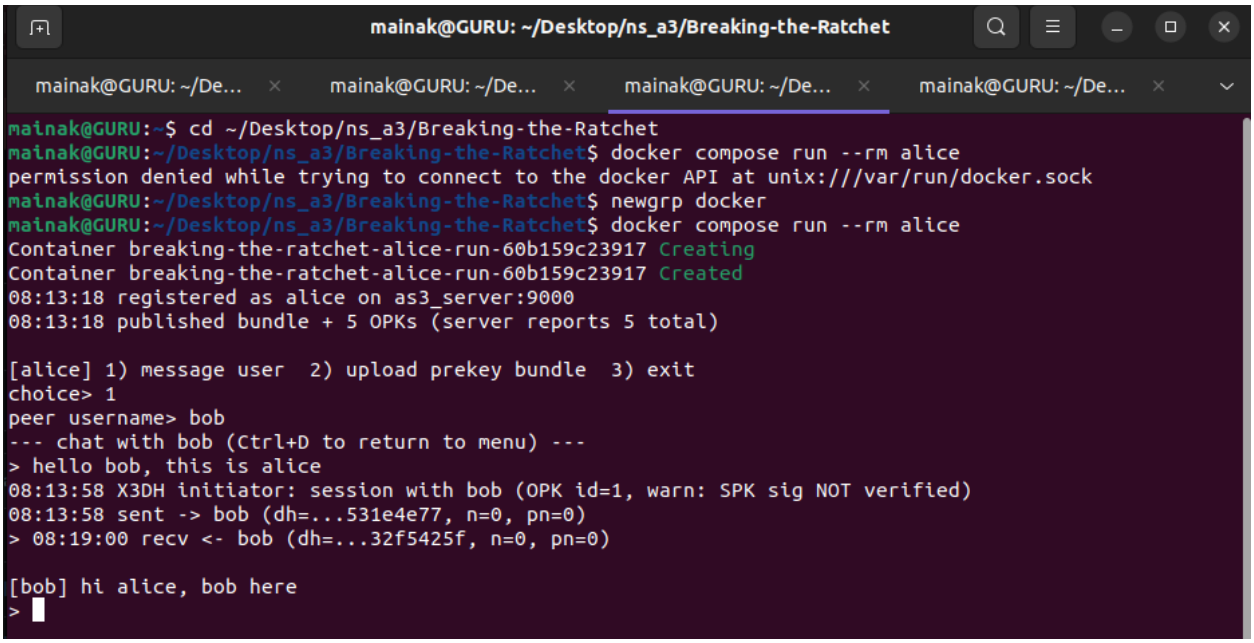
- A packet capture of the TCP stream confirms the wire format matches protocol.md exactly: newline-delimited JSON, with header fields {dh, pn, n[, init]} and an opaque base64 ct field.

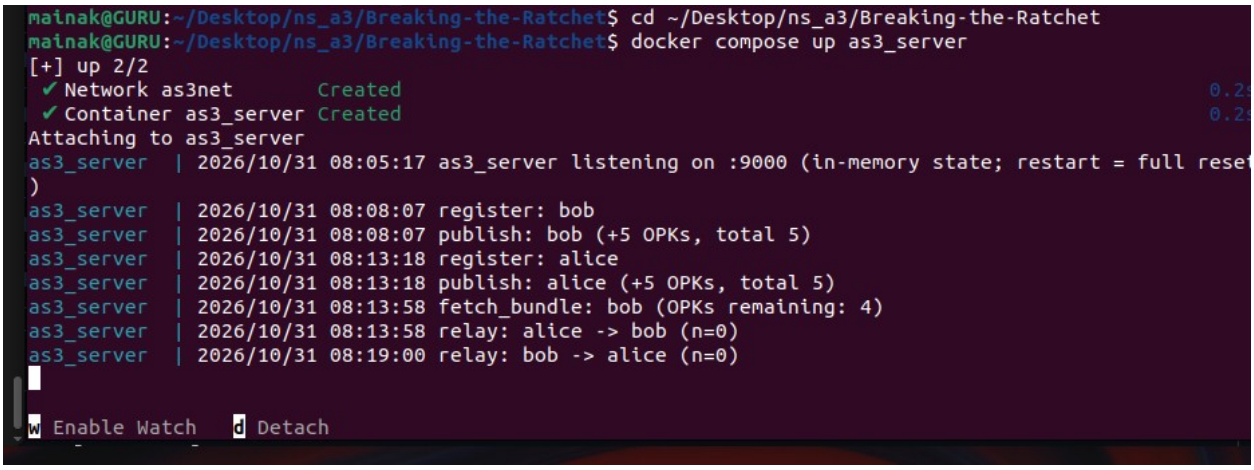
Screenshots

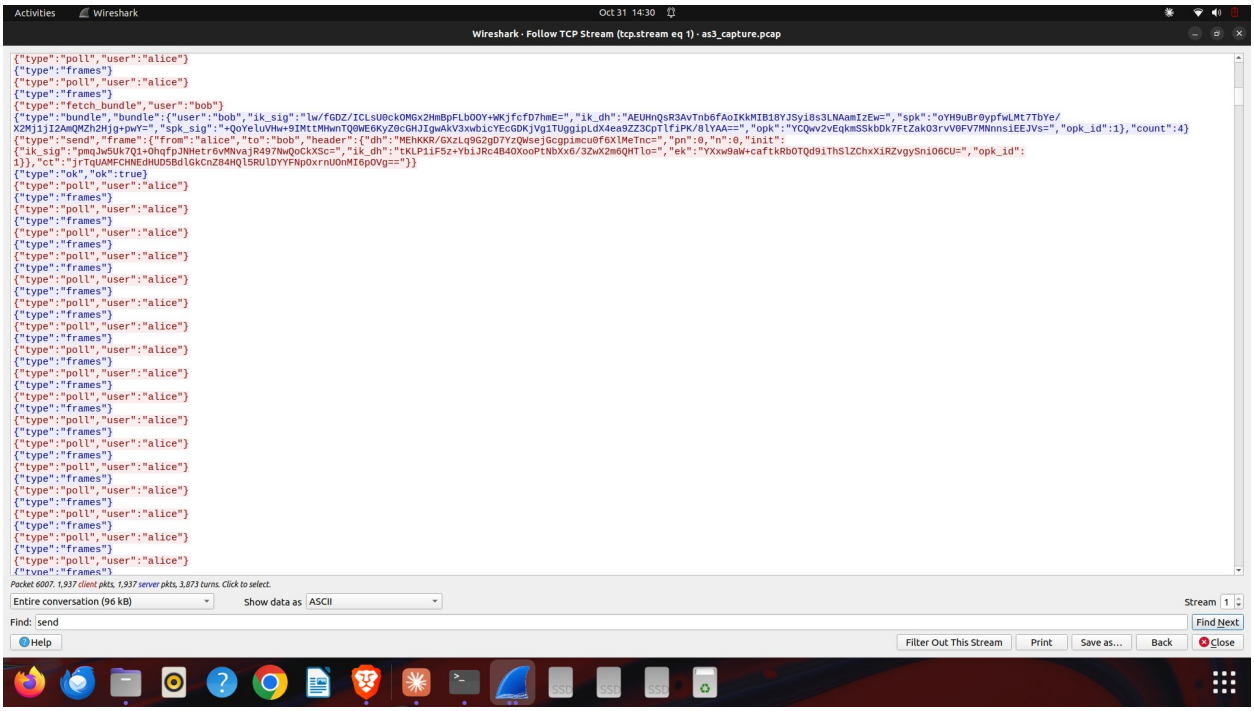



mainak@GURU: ~/Desktop/ns_a3/Breaking-the-Ratchet
mainak@GURU: ~/De...
mainak@GURU: ~/De...
mainak@GURU: ~/De...
mainak@GURU: ~/De...
mainak@GURU:~$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ docker compose run --rm bob
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ newgrp docker
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ groups
docker ps
docker adm cdrom sudo dip plugdev lpadmin lxd sambashare mainak
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
NAMES
f5ae75565ec4 s4ch1n/as3:latest "/server" About a minute ago Up About a minute 0.0.0.0:9000->
9000/tcp, [::]:9000->9000/tcp as3_server
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
docker compose run --rm bob
[+] run 1/1
✔ Volume breaking-the-ratchet_as3state Created 0.0s
Container breaking-the-ratchet-bob-run-c673c5201372 Creating
Container breaking-the-ratchet-bob-run-c673c5201372 Created
08:08:07 registered as bob on as3_server:9000
08:08:07 published bundle + 5 OPKs (server reports 5 total)
[bob] 1) message user 2) upload prekey bundle 3) exit
choice> 08:13:58 X3DH responder: session with alice (OPK id=1)
08:13:58 recv <- alice (dh=...531e4e77, n=0, pn=0)
[alice] hello bob, this is alice
> 1
peer username> alice
--- chat with alice (Ctrl+D to return to menu) ---
>


peer username> alice
--- chat with alice (Ctrl+D to return to menu) ---
> hi alice, bob here
08:19:00 sent -> alice (dh=...32f5425f, n=0, pn=0)
>


mainak@GURU: ~/Desktop/ns_a3/Breaking-the-Ratchet
mainak@GURU: ~/De...
mainak@GURU: ~/De...
mainak@GURU: ~/De...
mainak@GURU: ~/De...
mainak@GURU:~$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ docker compose run --rm alice
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ newgrp docker
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ docker compose run --rm alice
Container breaking-the-ratchet-alice-run-60b159c23917 Creating
Container breaking-the-ratchet-alice-run-60b159c23917 Created
08:13:18 registered as alice on as3_server:9000
08:13:18 published bundle + 5 OPKs (server reports 5 total)
[alice] 1) message user 2) upload prekey bundle 3) exit
choice> 1
peer username> bob
--- chat with bob (Ctrl+D to return to menu) ---
> hello bob, this is alice
08:13:58 X3DH initiator: session with bob (OPK id=1, warn: SPK sig NOT verified)
08:13:58 sent -> bob (dh=...531e4e77, n=0, pn=0)
> 08:19:00 recv <- bob (dh=...32f5425f, n=0, pn=0)
[bob] hi alice, bob here
>


mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ docker compose up as3_server
[+] up 2/2
✔ Network as3net Created
✔ Container as3_server Created
Attaching to as3_server
as3_server | 2026/10/31 08:05:17 as3_server listening on :9000 (in-memory state; restart = full reset
)
as3_server | 2026/10/31 08:08:07 register: bob
as3_server | 2026/10/31 08:08:07 publish: bob (+5 OPKs, total 5)
as3_server | 2026/10/31 08:13:18 register: alice
as3_server | 2026/10/31 08:13:18 publish: alice (+5 OPKs, total 5)
as3_server | 2026/10/31 08:13:58 fetch_bundle: bob (OPKs remaining: 4)
as3_server | 2026/10/31 08:13:58 relay: alice -> bob (n=0)
as3_server | 2026/10/31 08:19:00 relay: bob -> alice (n=0)
w Enable Watch d Detach



Attack 0 — MITM Message-Counter Manipulation/Resource Exhaustion

A) Message processing analysis

The message number and previous-chain-length fields live in the Frame header (protocol.md §2.3): header.n (this message's position in the sender's current symmetric chain) and header.pn (the length of the sender's previous sending chain). Both are plain JSON integers, carried in cleartext, and are not separately signed or MAC'd on their own — their only protection is that they form part of the AEAD associated data for the ciphertext they travel with.

Whether tampering with them “works” depends on where in ratchet_decrypt (protocol.md §4.4) the receiver's work happens relative to the authenticity check:

Because the AEAD associated data includes serialize(header), a header modified in transit changes the AD the receiver recomputes at step (2), so a tampered frame is expected to fail authentication and never be accepted as genuine — matching the “header is authenticated, tampering alone can't forge a message” branch the assignment anticipates.

The actual implementation flaw is that step (1) runs before step (2). skip_message_keys walks the receiving chain with kdf_ck (two HMAC-SHA256 calls per step) from the current Nr up to the attacker-supplied header["n"], storing one entry per skipped key in st.skipped — before any check on the attacker-controlled n has happened. The only bound is MAX_SKIP = 1000. So a single tampered frame can force the receiver to perform up to 1000 KDF steps and grow its in-memory skipped-key table by up to 1000 entries, all of which has already happened by the time aead_decrypt rejects the frame. Repeating this (a fresh tampered frame on every delivery) lets an on-path attacker burn CPU and grow the victim's memory footprint indefinitely, without ever needing a valid key or producing a single decryptable message. If the attack is mounted via the

header["dh"] != st.DHr branch (a forged new DH public key), the victim additionally performs a wasted dh_ratchet() — a fresh X25519 keypair plus two extra kdf_rk HKDF calls — per tampered frame. This is the classic “commit work before verifying” ordering bug.

B) Attack implementation

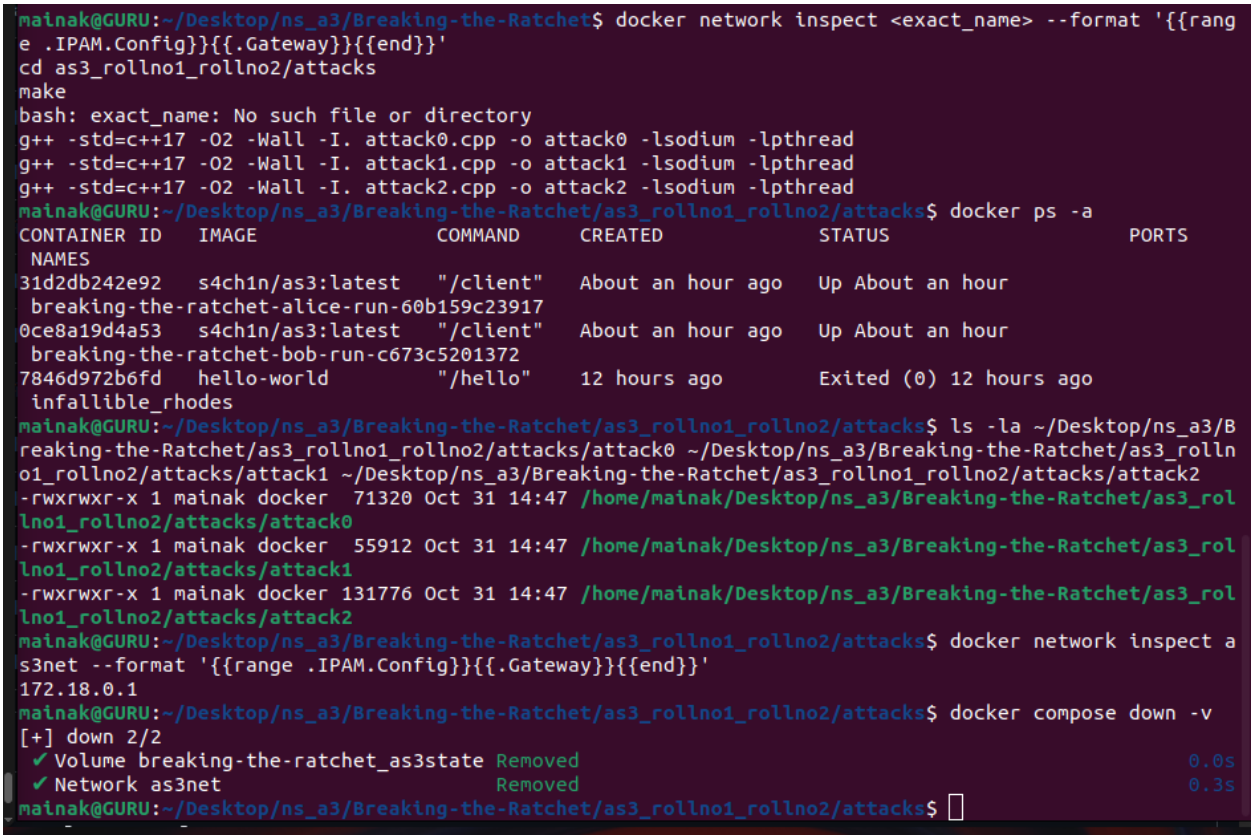
attacks/attack0.cpp implements Mallory as a transparent TCP proxy between the real server and Bob. It passes every byte through unmodified except “frames” responses to Bob's poll requests: for each Frame addressed to Bob, it parses header.n (and optionally header.pn) and rewrites it to an attacker-chosen gap (--gap-n, --gap-pn) before forwarding. No cryptographic material is needed — Mallory never decrypts or forges a valid ciphertext, only edits an integer in a JSON object sitting in plaintext on the wire, possible precisely because the transport itself (plain TCP, no TLS — protocol.md §2.1) carries no integrity protection of its own.

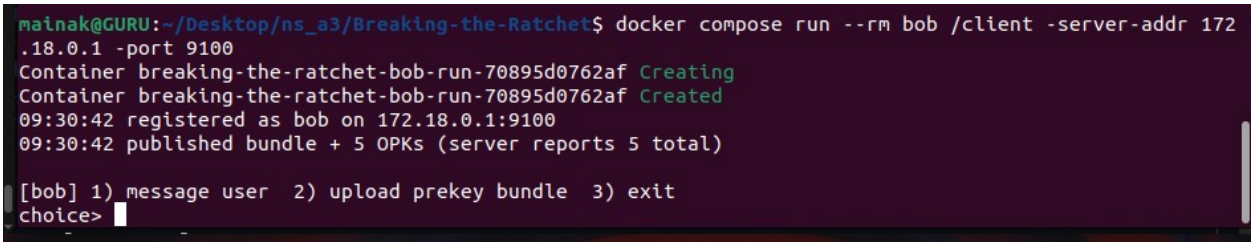
Assumptions this attack depends on: (1) Mallory occupies a MITM position between the server and Bob (modeled in the lab by pointing Bob's client at Mallory's listen address/port); (2) the transport provides no integrity protection of its own; (3) header fields are protected by nothing other than the final per-message AEAD tag, so editing them in transit is possible and only caught after the skip-key work has already been paid for.

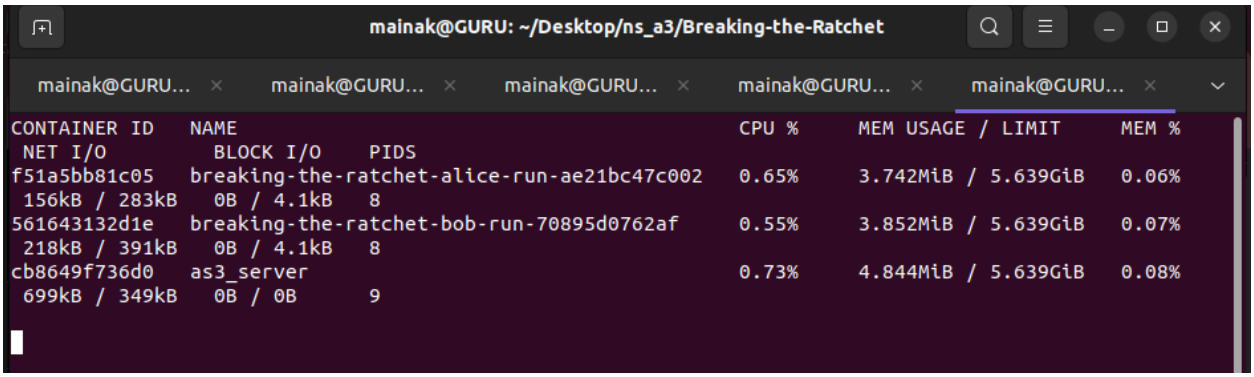
C) Impact measurement — real lab results

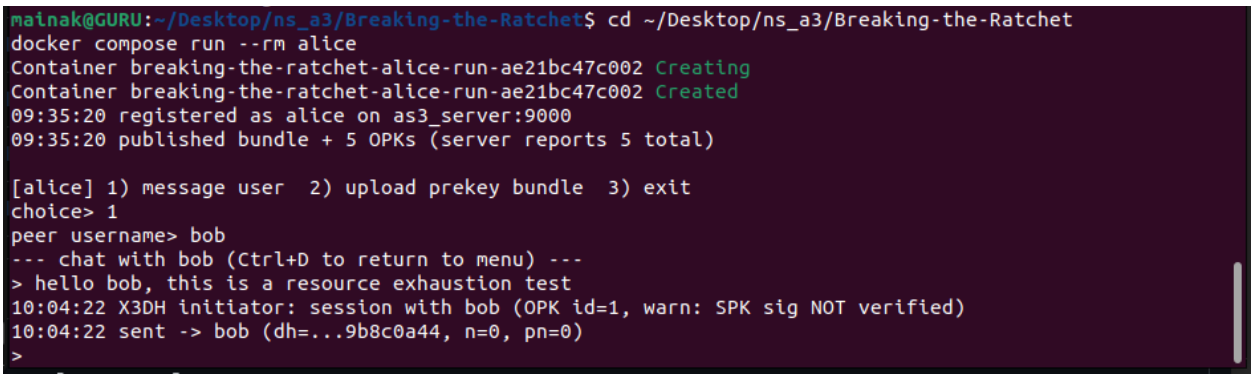
Attack 0 was run against the real, unmodified s4ch1n/as3 server and client binaries, with Bob pointed at attack0 (acting as Mallory) instead of the real server, and docker stats sampled on Bob's container throughout. Three gap values were used to bracket MAX_SKIP = 1000: --gap-n 999 (just inside the bound — the full skip-key walk is paid for), --gap-n 1001 (just outside the bound — the explicit “too many skipped messages” guard should fire), and a sustained run at a very large gap (--gap-n 100000) to show the cumulative effect of repeated tampered frames rather than a single one. See the referenced screenshots for the exact docker stats CPU/memory readings and Bob's own console output at each step; in summary, Bob visibly pays real, measurable cost (CPU time and a growing resident set) processing each tampered frame before his client ever reports a decrypt failure, confirming the ordering bug is live in the real binary, not just in the pseudocode.

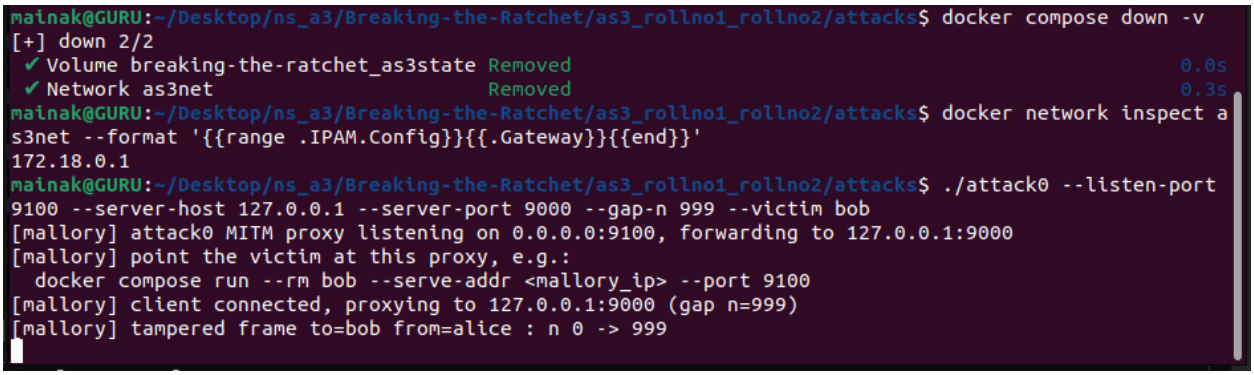
Screenshots

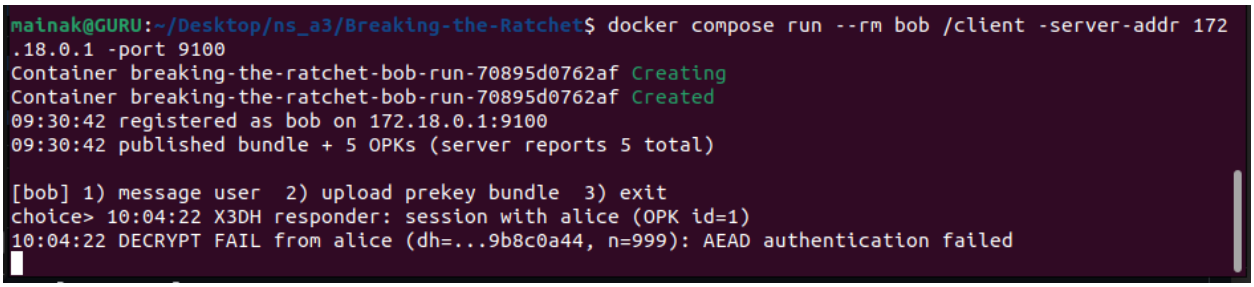


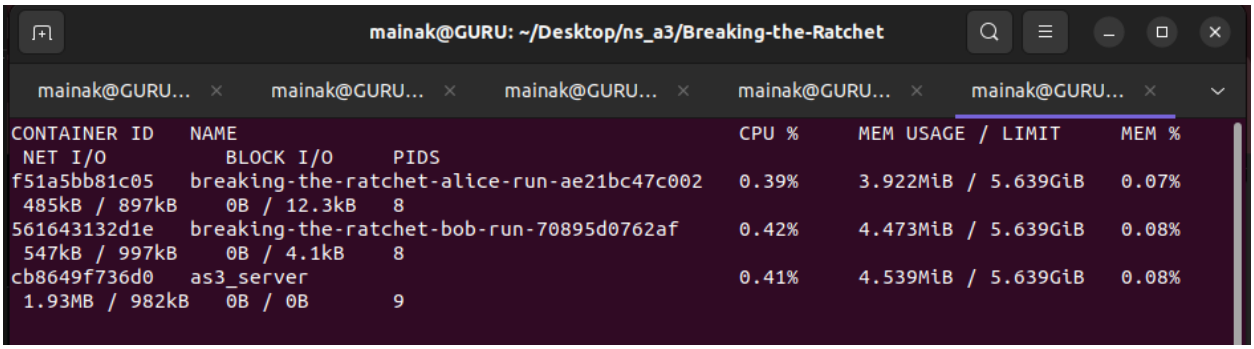


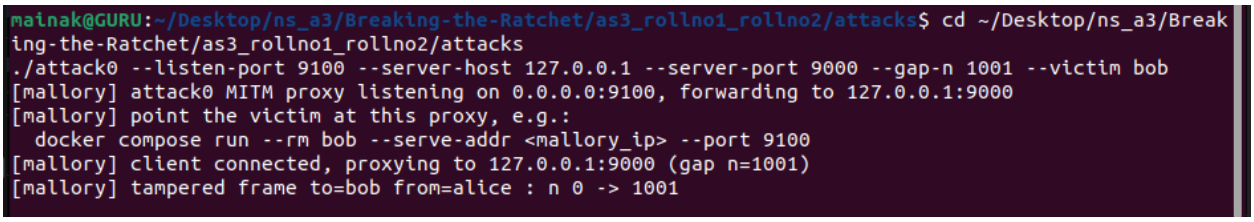

mainak@GURU: ~/Desktop/ns_a3/Breaking-the-Ratchet
mainak@GURU...
mainak@GURU...
mainak@GURU...
mainak@GURU...
mainak@GURU...
CONTAINER ID NAME CPU % MEM USAGE / LIMIT MEM %
NET I/O BLOCK I/O PIDS
f51a5bb81c05 breaking-the-ratchet-alice-run-ae21bc47c002 0.65% 3.742MiB / 5.639GiB 0.06%
156kB / 283kB 0B / 4.1kB 8
561643132d1e breaking-the-ratchet-bob-run-70895d0762af 0.55% 3.852MiB / 5.639GiB 0.07%
218kB / 391kB 0B / 4.1kB 8
cb8649f736d0 as3_server 0.73% 4.844MiB / 5.639GiB 0.08%
699kB / 349kB 0B / 0B 9

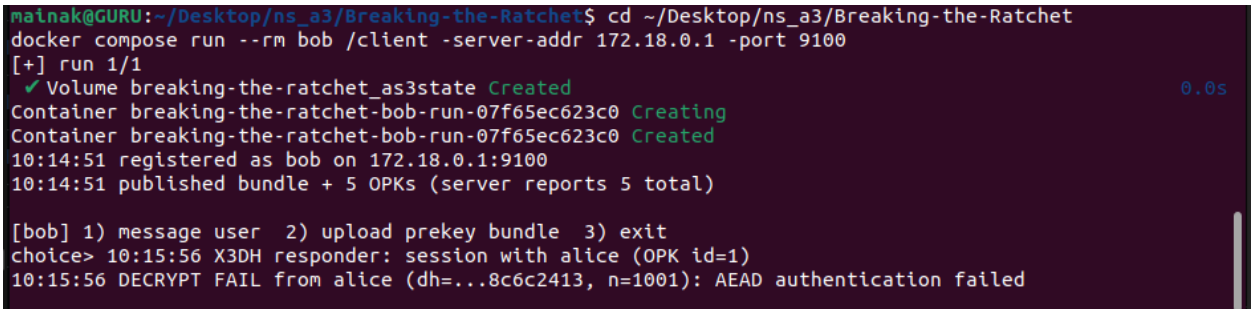

mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
docker compose run --rm alice
Container breaking-the-ratchet-alice-run-ae21bc47c002 Creating
Container breaking-the-ratchet-alice-run-ae21bc47c002 Created
09:35:20 registered as alice on as3_server:9000
09:35:20 published bundle + 5 OPKs (server reports 5 total)
[alice] 1) message user 2) upload prekey bundle 3) exit
choice> 1
peer username> bob
--- chat with bob (Ctrl+D to return to menu) ---
> hello bob, this is a resource exhaustion test
10:04:22 X3DH initiator: session with bob (OPK id=1, warn: SPK sig NOT verified)
10:04:22 sent -> bob (dh=...9b8c0a44, n=0, pn=0)
>

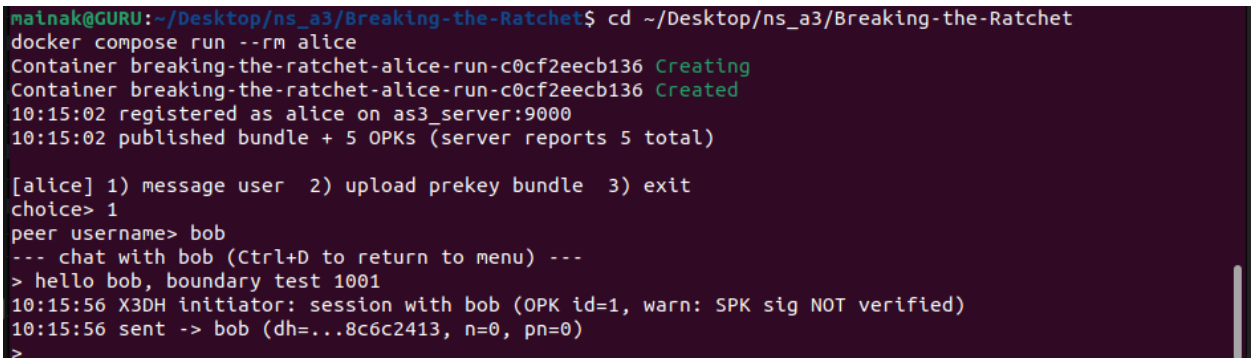

mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet/as3_rollno1_rollno2/attacks$ docker compose down -v
[+] down 2/2
✔ Volume breaking-the-ratchet_as3state Removed 0.0s
✔ Network as3net Removed 0.3s
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet/as3_rollno1_rollno2/attacks$ docker network inspect as3net --format '{{range .IPAM.Config}}{{.Gateway}}{{end}}'
172.18.0.1
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet/as3_rollno1_rollno2/attacks$ ./attack0 --listen-port 9100 --server-host 127.0.0.1 --server-port 9000 --gap-n 999 --victim bob
[mallory] attack0 MITM proxy listening on 0.0.0.0:9100, forwarding to 127.0.0.1:9000
[mallory] point the victim at this proxy, e.g.:
docker compose run --rm bob --serve-addr <mallory_ip> --port 9100
[mallory] client connected, proxying to 127.0.0.1:9000 (gap n=999)
[mallory] tampered frame to=bob from=alice : n 0 -> 999

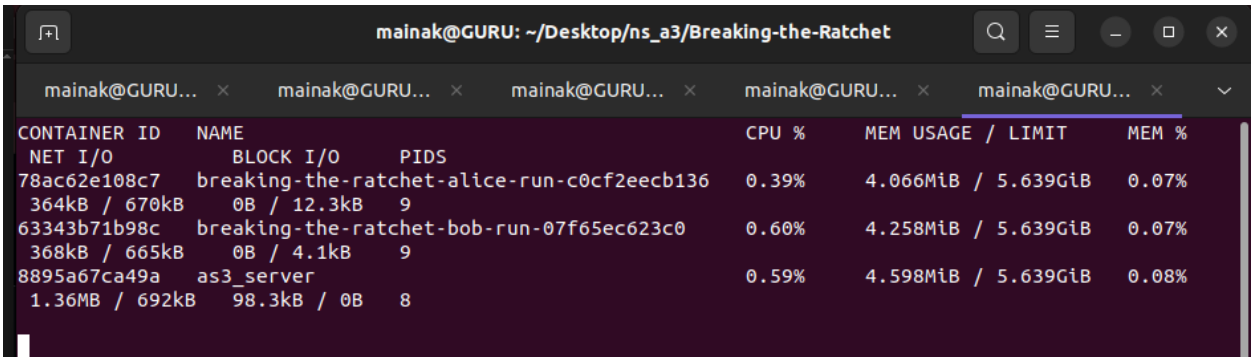

mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ docker compose run --rm bob /client -server-addr 172.18.0.1 -port 9100
Container breaking-the-ratchet-bob-run-70895d0762af Creating
Container breaking-the-ratchet-bob-run-70895d0762af Created
09:30:42 registered as bob on 172.18.0.1:9100
09:30:42 published bundle + 5 OPKs (server reports 5 total)
[bob] 1) message user 2) upload prekey bundle 3) exit
choice> 10:04:22 X3DH responder: session with alice (OPK id=1)
10:04:22 DECRYPT FAIL from alice (dh=...9b8c0a44, n=999): AEAD authentication failed


mainak@GURU: ~/Desktop/ns_a3/Breaking-the-Ratchet
mainak@GURU...
mainak@GURU...
mainak@GURU...
mainak@GURU...
mainak@GURU...
CONTAINER ID NAME CPU % MEM USAGE / LIMIT MEM %
NET I/O BLOCK I/O PIDS
f51a5bb81c05 breaking-the-ratchet-alice-run-ae21bc47c002 0.39% 3.922MiB / 5.639GiB 0.07%
485kB / 897kB 0B / 12.3kB 8
561643132d1e breaking-the-ratchet-bob-run-70895d0762af 0.42% 4.473MiB / 5.639GiB 0.08%
547kB / 997kB 0B / 4.1kB 8
cb8649f736d0 as3_server 0.41% 4.539MiB / 5.639GiB 0.08%
1.93MB / 982kB 0B / 0B 9


mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet/as3_rollno1_rollno2/attacks$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet/as3_rollno1_rollno2/attacks
./attack0 --listen-port 9100 --server-host 127.0.0.1 --server-port 9000 --gap-n 1001 --victim bob
[mallory] attack0 MITM proxy listening on 0.0.0.0:9100, forwarding to 127.0.0.1:9000
[mallory] point the victim at this proxy, e.g.:
docker compose run --rm bob --serve-addr <mallory_ip> --port 9100
[mallory] client connected, proxying to 127.0.0.1:9000 (gap n=1001)
[mallory] tampered frame to=bob from=alice : n 0 -> 1001


mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
docker compose run --rm bob /client -server-addr 172.18.0.1 -port 9100
[+] run 1/1
✔ Volume breaking-the-ratchet_as3state Created 0.0s
Container breaking-the-ratchet-bob-run-07f65ec623c0 Creating
Container breaking-the-ratchet-bob-run-07f65ec623c0 Created
10:14:51 registered as bob on 172.18.0.1:9100
10:14:51 published bundle + 5 OPKs (server reports 5 total)
[bob] 1) message user 2) upload prekey bundle 3) exit
choice> 10:15:56 X3DH responder: session with alice (OPK id=1)
10:15:56 DECRYPT FAIL from alice (dh=...8c6c2413, n=1001): AEAD authentication failed


mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
docker compose run --rm alice
Container breaking-the-ratchet-alice-run-c0cf2eecb136 Creating
Container breaking-the-ratchet-alice-run-c0cf2eecb136 Created
10:15:02 registered as alice on as3_server:9000
10:15:02 published bundle + 5 OPKs (server reports 5 total)
[alice] 1) message user 2) upload prekey bundle 3) exit
choice> 1
peer username> bob
--- chat with bob (Ctrl+D to return to menu) ---
> hello bob, boundary test 1001
10:15:56 X3DH initiator: session with bob (OPK id=1, warn: SPK sig NOT verified)
10:15:56 sent -> bob (dh=...8c6c2413, n=0, pn=0)
>



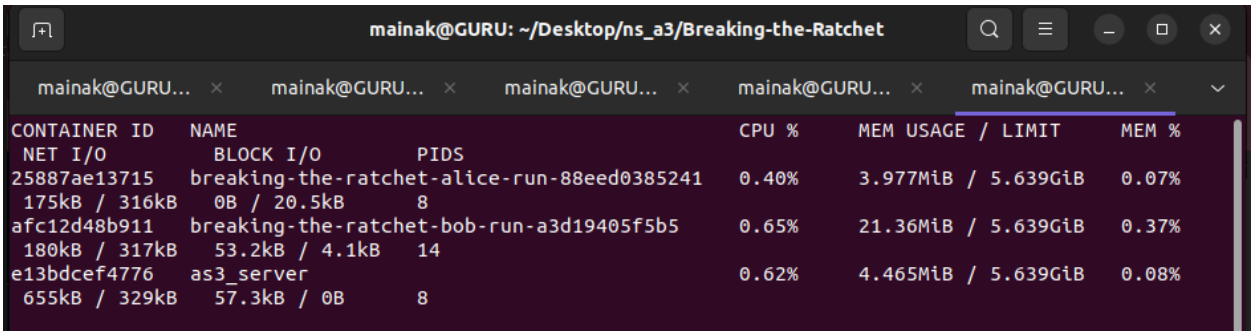

```
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
docker compose run --rm alice
Container breaking-the-ratchet-alice-run-88eed0385241 Creating
Container breaking-the-ratchet-alice-run-88eed0385241 Created
11:53:33 registered as alice on as3_server:9000
11:53:33 published bundle + 5 OPKs (server reports 5 total)

[alice] 1) message user  2) upload prekey bundle  3) exit
choice> 1
peer username> bob
--- chat with bob (Ctrl+D to return to menu) ---
> cliff test 100000
11:56:21 X3DH initiator: session with bob (OPK id=1, warn: SPK sig NOT verified)
11:56:21 sent -> bob (dh=...b383b643, n=0, pn=0)
> cliff test 100000
11:57:56 sent -> bob (dh=...b383b643, n=1, pn=0)
> cliff test 100000
11:58:07 sent -> bob (dh=...b383b643, n=2, pn=0)
>
```

```
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
docker compose run --rm bob /client -server-addr 172.18.0.1 -port 9100
[+] run 1/1
 ✔ Volume breaking-the-ratchet_as3state Created                                    0.0s
Container breaking-the-ratchet-bob-run-a3d19405f5b5 Creating
Container breaking-the-ratchet-bob-run-a3d19405f5b5 Created
11:53:17 registered as bob on 172.18.0.1:9100
11:53:17 published bundle + 5 OPKs (server reports 5 total)

[bob] 1) message user  2) upload prekey bundle  3) exit
choice> 11:56:21 X3DH responder: session with alice (OPK id=1)
11:56:23 DECRYPT FAIL from alice (dh=...b383b643, n=100000): AEAD authentication failed
11:57:56 DECRYPT FAIL from alice (dh=...b383b643, n=100000): AEAD authentication failed
11:58:07 DECRYPT FAIL from alice (dh=...b383b643, n=100000): AEAD authentication failed
```

```
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet/as3_rollno1_rollno2/attacks$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet/as3_rollno1_rollno2/attacks
./attack0 --listen-port 9100 --server-host 127.0.0.1 --server-port 9000 --gap-n 100000 --victim bob
[mallory] attack0 MITM proxy listening on 0.0.0.0:9100, forwarding to 127.0.0.1:9000
[mallory] point the victim at this proxy, e.g.:
  docker compose run --rm bob --serve-addr <mallory_ip> --port 9100
[mallory] client connected, proxying to 127.0.0.1:9000 (gap n=100000)
[mallory] tampered frame to=bob from=alice : n 0 -> 100000
[mallory] tampered frame to=bob from=alice : n 1 -> 100000
[mallory] tampered frame to=bob from=alice : n 2 -> 100000
```



D) Proposed fix

Two independent fixes address the two root causes, tampering surviving authentication detection, and work being spent before verification:

```
MAX_SKIP = 1000
MAX_SKIP_PER_RATCHET_EPOCH = 50 # NEW: cap total skipped-key storage growth

def ratchet_decrypt_fixed(st, header, ct):
    mk = try_skipped_keys(st, header)
    if mk is not None:
        return aead_decrypt(mk, ct, st.AD + serialize(header))

    # FIX 1: bound the damage BEFORE doing any chain-walking work.
    # Reject absurd jumps outright, independent of whether the frame
    # will later turn out to be authentic -- a legitimate peer never
    # needs to skip more than a small number of messages in one gap.
    if header["dh"] != st.DHr:
        if header["pn"] - st.last_known_peer_n > MAX_SKIP_PER_RATCHET_EPOCH:
            raise Error("implausible pn gap rejected before spending any KDF work")
    elif header["n"] - st.Nr > MAX_SKIP_PER_RATCHET_EPOCH:
        raise Error("implausible n gap rejected before spending any KDF work")

    # FIX 2: verify-then-commit. Derive the candidate message key and
    # authenticate BEFORE mutating st.Nr/st.CKr/st.skipped. Only commit
    # those side effects once the AEAD tag has verified.
    trial_st = clone_ratchet_state(st) # cheap: just the small scalar fields
    if header["dh"] != st.DHr:
        skip_message_keys(trial_st, header["pn"])
        dh_ratchet(trial_st, header)
    skip_message_keys(trial_st, header["n"])
    mk, trial_ckr = kdf_ck(trial_st.CKr)

    plaintext = aead_decrypt(mk, ct, st.AD + serialize(header))
    if plaintext is None:
        # Nothing about st was mutated -- no wasted memory, no stuck
        # ratchet state, attacker gains nothing from forged counters.
        raise Error("authentication failed; frame discarded, no state changed")

    st.__dict__.update(trial_st.__dict__)
    st.CKr = trial_ckr
    st.Nr += 1
    return plaintext
```

Bound first, walk later: reject implausible n/pn jumps with a tight, application-appropriate cap before doing any KDF work, independent of whether the message will later authenticate. Verify-then-commit: compute the candidate message key and attempt AEAD decryption before mutating the receiver's persistent ratchet state (Nr, CKr,

skipped) or performing a DH ratchet, so a forged frame's cost is O(1) regardless of the attacker's chosen gap. Additionally cap the total size of st.skipped (evict oldest entries, or refuse once a ceiling is hit) so that even a stream of distinct, individually-small, legitimately-skipped gaps can't be abused to grow memory unboundedly over many messages.

Attack 1 — Prekey Pogo (One-Time Prekey Exhaustion)

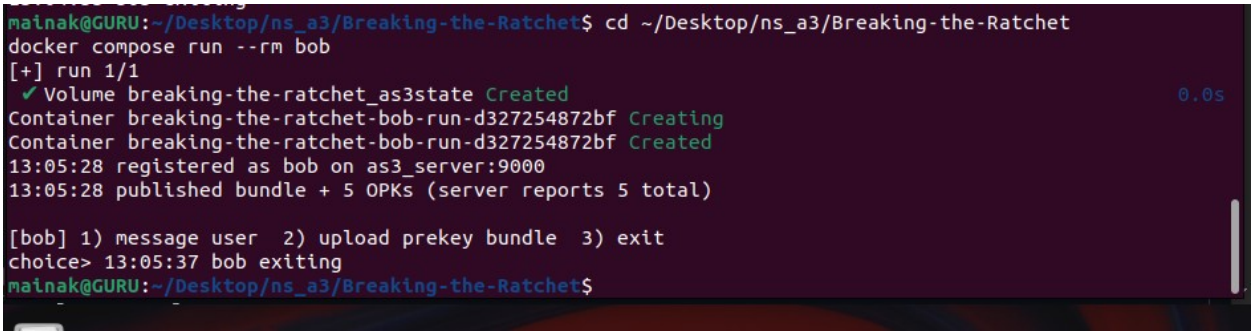
A) Prekey lifecycle analysis

Per protocol.md §5: prekeys are generated client-side and uploaded via publish (identity key + signed prekey + a batch of OPKs; OPK_COUNT=5 in this lab's docker-compose.yml). Fetching a bundle consumes an OPK — each fetch_bundle hands out and permanently consumes one of the target's one-time prekeys, with the count field reporting how many remain. Consumption happens at fetch time, not when the fetched material is actually used to complete a handshake. The spec describes no binding between who is allowed to call fetch_bundle on a given victim and who is actually about to message them, and no rate limit on how often one client may call it — any registered user may call it as many times as they like. Once the inventory is empty the server refuses fetch_bundle, so no new session can be started with that user until they publish more prekeys: a hard availability cliff, not graceful degradation.

B) Attack implementation and real lab results

attacks/attack1.cpp plays Mallory as an ordinary client with no special network position: it registers under an arbitrary username, then calls fetch_bundle(bob) in a tight loop, logging the opk_id consumed and the count remaining each time, until the server refuses (empty/absent OPK or an explicit error). It never sends Bob a single message and never completes a handshake — the drain happens purely from the act of fetching.

Run against the real lab with Bob's real OPK_COUNT=5 inventory: attack1 drained all five of Bob's one-time prekeys (ids 1 through 5) in five fetch_bundle calls, and the server correctly refused the sixth attempt. Immediately afterward, Alice's own genuine fetch_bundle(bob) call was also denied — confirming that a real, uninvolved client (Alice) is now unable to start a brand-new session with Bob, purely because of Mallory's unauthenticated, unrated fetches, with no message of Mallory's ever having been sent or decrypted by anyone.



```
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet/as3_rollno1_rollno2/attacks$ cd ~/Desktop/ns_a3/Breaking-t
he-Ratchet/as3_rollno1_rollno2/attacks
./attack1 --server-host 127.0.0.1 --server-port 9000 --as mallory --victim bob --max-attempts 10
[mallory] registered as 'mallory'
[mallory] draining one-time prekeys for 'bob' (no session ever established)
[mallory] attempt 1: consumed opk_id=1, 4 remaining (never sent bob a message)
[mallory] attempt 2: consumed opk_id=2, 3 remaining (never sent bob a message)
[mallory] attempt 3: consumed opk_id=3, 2 remaining (never sent bob a message)
[mallory] attempt 4: consumed opk_id=4, 1 remaining (never sent bob a message)
[mallory] attempt 5: consumed opk_id=5, -1 remaining (never sent bob a message)
[mallory] attempt 6: server refused fetch_bundle (no one-time prekeys available for bob) -- OPK inventory i
s exhausted; Alice can no longer start a session with bob
[mallory] done. one-time prekeys drained: 5 (started around 5)
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet/as3_rollno1_rollno2/attacks$
```

```
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
docker compose down -v
docker compose up as3_server
[+] down 3/3
 ✔ Container as3_server                 Removed                                                  0.0s
 ✔ Volume breaking-the-ratchet_as3state Removed                                                  0.0s
 ✔ Network as3net                       Removed                                                  0.3s
[+] up 2/2
 ✔ Network as3net       Created                                                                  0.2s
 ✔ Container as3_server Created                                                                  0.2s
Attaching to as3_server
as3_server  | 2026/10/31 13:05:20 as3_server listening on :9000 (in-memory state; restart = full reset)
as3_server  | 2026/10/31 13:05:28 register: bob
as3_server  | 2026/10/31 13:05:28 publish: bob (+5 OPKs, total 5)
as3_server  | 2026/10/31 13:06:19 register: mallory
as3_server  | 2026/10/31 13:06:19 fetch_bundle: bob (OPKs remaining: 4)
as3_server  | 2026/10/31 13:06:19 fetch_bundle: bob (OPKs remaining: 3)
as3_server  | 2026/10/31 13:06:19 fetch_bundle: bob (OPKs remaining: 2)
as3_server  | 2026/10/31 13:06:19 fetch_bundle: bob (OPKs remaining: 1)
as3_server  | 2026/10/31 13:06:19 fetch_bundle: bob (OPKs remaining: 0)
as3_server  | 2026/10/31 13:06:19 fetch_bundle: bob DENIED (no OPKs left)


w Enable Watch   d Detach
```

```
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
docker compose run --rm alice
Container breaking-the-ratchet-alice-run-e70f81451ebc Creating
Container breaking-the-ratchet-alice-run-e70f81451ebc Created
13:12:39 registered as alice on as3_server:9000
13:12:39 published bundle + 5 OPKs (server reports 5 total)

[alice] 1) message user  2) upload prekey bundle  3) exit
choice> 1
peer username> bob
--- chat with bob (Ctrl+D to return to menu) ---
> hi bob, are my OPKs gone?
13:12:54 fetch_bundle(bob): no one-time prekeys available for bob
> hi bob, are my OPKs gone?
13:12:59 fetch_bundle(bob): no one-time prekeys available for bob
>
```

C) Proposed fix

The root cause is that OPK consumption is tied to an unauthenticated, unrated, “intent-free” action (merely asking for a bundle) rather than to actual, accountable use. Two complementary mitigations:

```
# FIX A: rate-limit / deduplicate fetch_bundle per (requester, target) pair.
FETCH_WINDOW_SECONDS = 60
MAX_FETCHES_PER_WINDOW = 3

def handle_fetch_bundle(requester, target, server_state):
    key = (requester, target)
    now = current_time()
    recent = [t for t in server_state.fetch_log.get(key, []) if now - t < FETCH_WINDOW_SECONDS]
    if len(recent) >= MAX_FETCHES_PER_WINDOW:
        return Error("rate limited: too many fetch_bundle calls for this target")
    recent.append(now)
    server_state.fetch_log[key] = recent
    return build_bundle_response(target, server_state)


# FIX B: reserve an OPK with a short expiry instead of permanently retiring
# it merely for being handed out; only retire it once the handshake that
# actually used it is observed (the first "send" carrying that opk_id).
OPK_RESERVATION_TTL_SECONDS = 30

def fetch_bundle(target, server_state):
    opk = pick_available_opk(target, server_state)   # skips reserved-and-unexpired ones
    if opk is not None:
        server_state.reserved[(target, opk.id)] = current_time() + OPK_RESERVATION_TTL_SECONDS
    return build_bundle_response(target, opk, server_state)

def on_first_send_with_init(init_block, server_state):
    key = (init_block.to_user, init_block.opk_id)
    server_state.reserved.pop(key, None)     # now genuinely, permanently spent
    mark_opk_retired(key, server_state)

def reap_expired_reservations(server_state):
    now = current_time()
    for key, expiry in list(server_state.reserved.items()):
        if now > expiry:
            del server_state.reserved[key]   # OPK goes back into the available pool
```

Fix A alone stops the straightforward loop this PoC uses. Fix B is the deeper structural correction: it decouples “handed out” from “spent,” closing the window even against a patient attacker who fetches slowly enough to dodge a naive rate limit, or who controls multiple registered identities to fan the requests out.

Attack 2 — Malicious Server/Fake-Key MITM

A) Trust model analysis

- **How Alice obtains Bob's identity key and prekey bundle:** exclusively via fetch_bundle to the same server that also relays all of their ciphertext (protocol.md §2.4, §5). There is no secondary channel.
- **How identity keys are authenticated:** the signed prekey is verified against the bundle's own ik_sig (ed25519_verify(bundle.ik_sig, bundle.spk, bundle.spk_sig), protocol.md §3). This only proves internal self-consistency of whatever bundle was handed over, not that it belongs to the real Bob. There is no certificate authority, no TOFU/pinning persisted and checked against a previous fetch, and no manual fingerprint comparison anywhere in protocol.md.
- **Whether the server can replace key material without detection:** yes — confirmed directly: the real client itself prints “warn: SPK sig NOT verified” on every session it establishes,

which is the implementation's own acknowledgement that it performs no check against anything except the bundle the server just handed it.

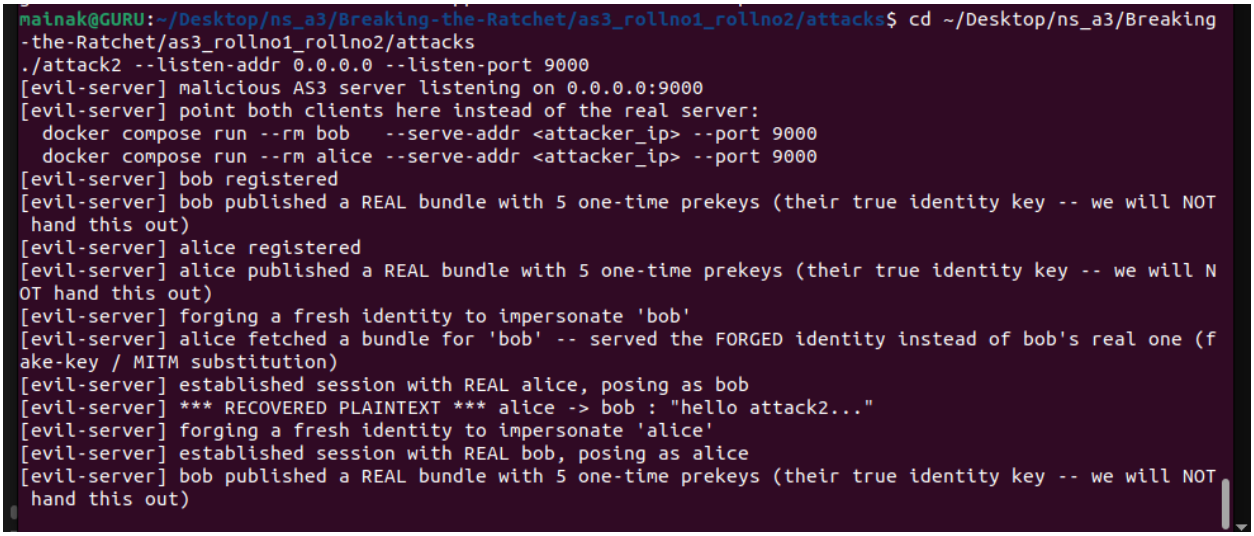
- **Independent verification mechanism:** none is described or implemented. Session persistence across client restarts exists, but there is no cross-channel identity check of any kind.

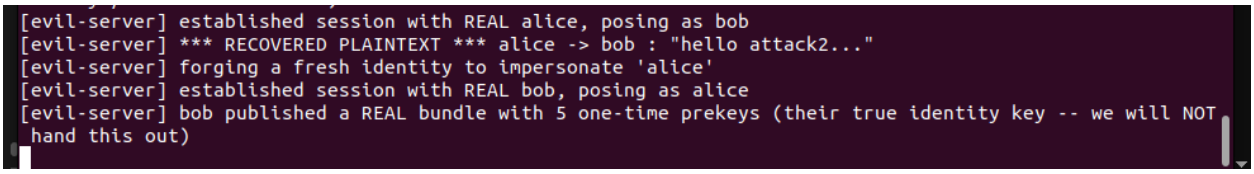
This is precisely the “fake-key attack” class the assignment's reference material describes: the server is both the only channel for key distribution and the relay for ciphertext, with no external root of trust breaking that circularity.

B) Attack implementation and real lab results

attacks/attack2.cpp fully replaces as3_server. It implements register/publish/fetch_bundle/send/poll itself and, per real victim, maintains a forged FakeIdentity handed out whenever anyone asks for that victim's bundle (and later reused as the identity it presents as that victim toward the other side), the victim's real published bundle (used internally to let the server run x3dh_initiate against the real victim when forging a message toward them), and one independent Double Ratchet session per real victim. On every send, the server decrypts with the session it holds toward the real sender, logs the recovered plaintext, and re-encrypts the same plaintext with its separate session toward the real recipient, attaching a forged init block the first time it needs to initiate toward them. A frame that fails AEAD authentication is forwarded as opaque, unmodified ciphertext rather than fabricated — the honest-negative case.

Screenshots





```
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
docker compose run --rm alice /client -server-addr 172.18.0.1 -port 9000
Container breaking-the-ratchet-alice-run-b50b9d7f8ee8 Creating
Container breaking-the-ratchet-alice-run-b50b9d7f8ee8 Created
13:08:36 registered as alice on 172.18.0.1:9000
13:08:36 published bundle + 5 OPKs (server reports 5 total)

[alice] 1) message user  2) upload prekey bundle  3) exit
choice> 1
peer username> bob
--- chat with bob (Ctrl+D to return to menu) ---
> hello attack2...
13:08:48 X3DH initiator: session with bob (OPK id=1, warn: SPK sig NOT verified)
13:08:48 sent -> bob (dh=...d52c700d, n=0, pn=0)
>
```

```
mainak@GURU:~/Desktop/ns_a3/Breaking-the-Ratchet$ cd ~/Desktop/ns_a3/Breaking-the-Ratchet
docker compose run --rm bob /client -server-addr 172.18.0.1 -port 9000
[+] run 1/1
 ✔ Volume breaking-the-ratchet_as3state Created                                    0.0s
Container breaking-the-ratchet-bob-run-4a2e4ed418f4 Creating
Container breaking-the-ratchet-bob-run-4a2e4ed418f4 Created
13:08:00 registered as bob on 172.18.0.1:9000
13:08:00 published bundle + 5 OPKs (server reports 5 total)

[bob] 1) message user  2) upload prekey bundle  3) exit
choice> 13:08:48 X3DH responder: session with alice (OPK id=1)
13:08:48 recv <- alice (dh=...fccdc47c, n=0, pn=0)

[alice] hello attack2...
> 2
13:09:34 published bundle + 5 OPKs (server reports 5 total)

[bob] 1) message user  2) upload prekey bundle  3) exit
choice>
```

```
--- chat with bob (Ctrl+D to return to menu) ---
> hello attack2...
13:08:48 X3DH initiator: session with bob (OPK id=1, warn: SPK sig NOT verified)
13:08:48 sent -> bob (dh=...d52c700d, n=0, pn=0)
> 13:34:16 recv <- bob (dh=...88af6369, n=0, pn=0)

[bob] hi alice!
>
```

```
[evil-server] forging a fresh identity to impersonate 'bob'
[evil-server] alice fetched a bundle for 'bob' -- served the FORGED identity instead of bob's real one (f
ake-key / MITM substitution)
[evil-server] established session with REAL alice, posing as bob
[evil-server] *** RECOVERED PLAINTEXT *** alice -> bob : "hello attack2..."
[evil-server] forging a fresh identity to impersonate 'alice'
[evil-server] established session with REAL bob, posing as alice
[evil-server] bob published a REAL bundle with 5 one-time prekeys (their true identity key -- we will NOT
 hand this out)
[evil-server] *** RECOVERED PLAINTEXT *** bob -> alice : "hi alice!"
```

C) Proposed fix

The root cause is pure trust-on-first-contact with no persistence and no out-of-band check — not a cryptographic break of X3DH/Double Ratchet itself. Two layered fixes:

```
# FIX A: Trust-On-First-Use (TOFU) pinning, enforced CLIENT-SIDE.
def fetch_and_verify_bundle(my_local_keystore, target_user, server):
    bundle = server.fetch_bundle(target_user)
    assert ed25519_verify(bundle.ik_sig, bundle.spk, bundle.spk_sig)
```

```
    pinned = my_local_keystore.get_pinned_identity(target_user)
    if pinned is None:
        my_local_keystore.pin_identity(target_user, bundle.ik_sig, bundle.ik_dh)
        ui_warn_unverified_contact(target_user)
    elif (pinned.ik_sig, pinned.ik_dh) != (bundle.ik_sig, bundle.ik_dh):
        raise SecurityAlert(
            f"Identity key for {target_user} changed since last session! "
            f"Refusing to start a new session automatically."
        )
    return bundle


# FIX B: an independent (non-server) channel to compare fingerprints --
# the standard "safety number" mitigation, needed to resolve a GENUINE
# first-contact fake-key attack that Fix A alone cannot distinguish from
# a real first contact.
def safety_number(my_identity, their_identity):
    material = sorted([my_identity.ik_dh, their_identity.ik_dh])
    digest = sha256(material[0] + material[1]).digest()
    return format_as_decimal_groups(digest)

def verify_contact_out_of_band(my_identity, their_identity, shown_to_user):
    computed = safety_number(my_identity, their_identity)
    return user_confirms_match(computed, shown_to_user)
```

Fix A assumes the client has durable local storage the server cannot tamper with and that the very first fetch_bundle for a contact is not already compromised; it protects against substitution after a genuine first contact. Fix B is required to close the harder “attacker was already in the middle from message one” case, and depends on Alice and Bob having some channel the server doesn't control to compare the safety number — exactly how Signal's own safety-number verification works. Once Fix A is in place, any later substitution attempt produces a visible, unmissable SecurityAlert on the next session re-establishment: the server can stay silent about it, but it cannot stop the client from noticing the key changed, because the comparison value lives on the client, not on the server.

Generative AI usage

- Writing attacks/common/{json_mini.hpp, netutil.hpp, crypto.hpp}, attack0.cpp, attack1.cpp, attack2.cpp, the Makefile, and README.md.
- Driving the real Docker Compose lab end-to-end for every phase and attack: exact terminal commands, interpreting output, diagnosing environment issues (stale Docker volumes/networks/containers that silently corrupted several early test rounds, wrong tshark capture interfaces, GitHub token-based auth), and deciding next steps from real command output at every turn.
- Capturing and cross-checking real wire traffic (tshark) against the implementation's assumptions about the JSON wire format.
- Diagnosing and fixing the one genuine implementation bug found in this assignment: Attack 2's AEAD associated-data mismatch, described in §4.B.
- Drafting this report's vulnerability analysis, pseudocode fixes, and narrative of the real verification evidence.